

MEDCORE HMS

Technical Architecture, Roadmap & Tooling

Modular HMS — free & open-source stack with pinned versions and install links

Team of 2 • ~12 months • Pilot-ready core + OPD by month 4 • Version 1.0 • July 2026

Contents

1. Architecture Overview

MedCore HMS is a modular monolith with a plugin system — one deployable application whose departments and features are pluggable packages. This is deliberately NOT microservices: for a two-person team deploying single-tenant on each hospital's own server, a modular monolith is far simpler to build, deploy, and support, while still giving true install-per-package modularity. The proven reference for this model is Odoo, a free/open-source business platform extended by installable apps.

One Software — Modular Package Architecture

Modular monolith with a plugin system: install only the packages a hospital needs

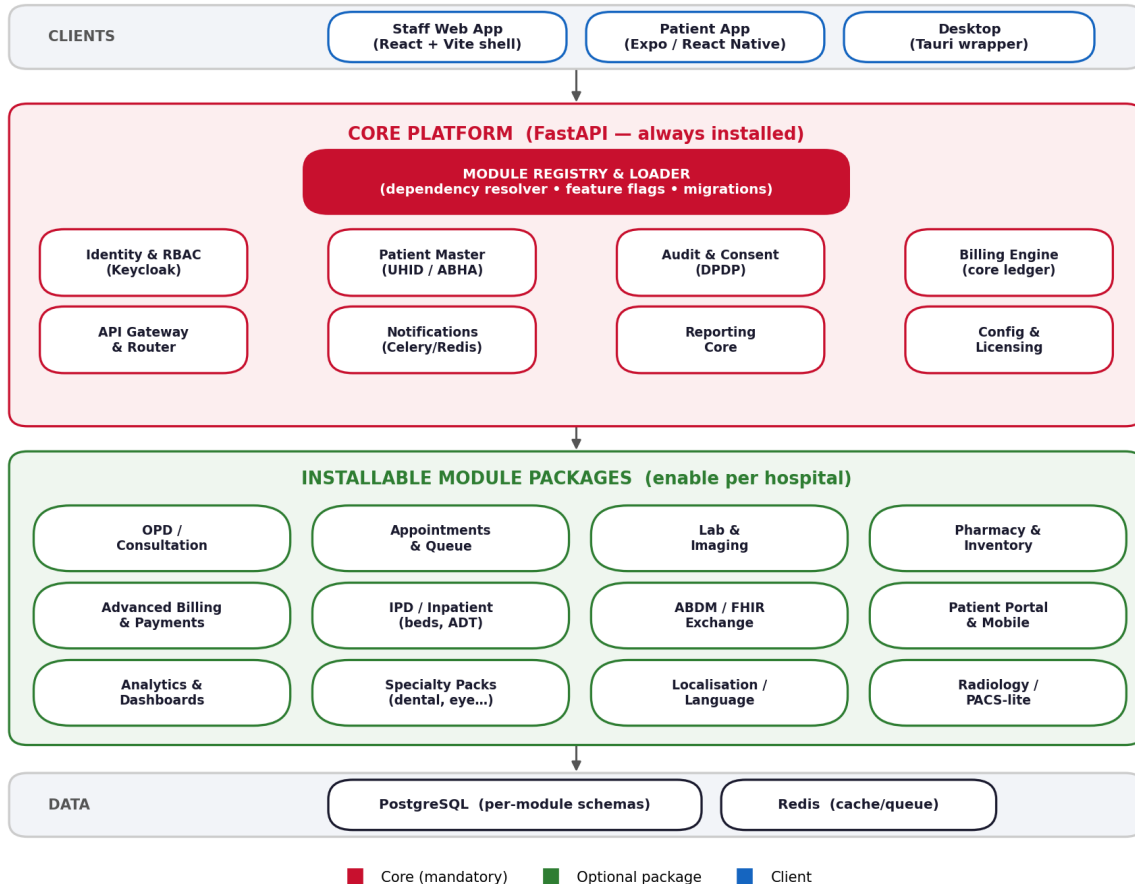


Figure 1 — Core platform with departments/features as installable packages.

Architecture decision: Modular monolith + plugin system. One codebase, one deployable, one database, one login; packages enabled per hospital. Revisit a managed-cloud/multi-tenant option only after several installs exist.

2. Deployment Topology

Every hospital runs the same stack on its own server via Docker Compose. Caddy terminates HTTPS and proxies to the FastAPI core; the core loads whichever packages are enabled; PostgreSQL, Keycloak, and Valkey run as containers alongside. Clients connect over the hospital LAN, so the system keeps working without internet.

Deployment Topology — One Hospital Server (Single-Tenant)

Everything runs on the hospital's own server via Docker Compose; clients connect over the LAN

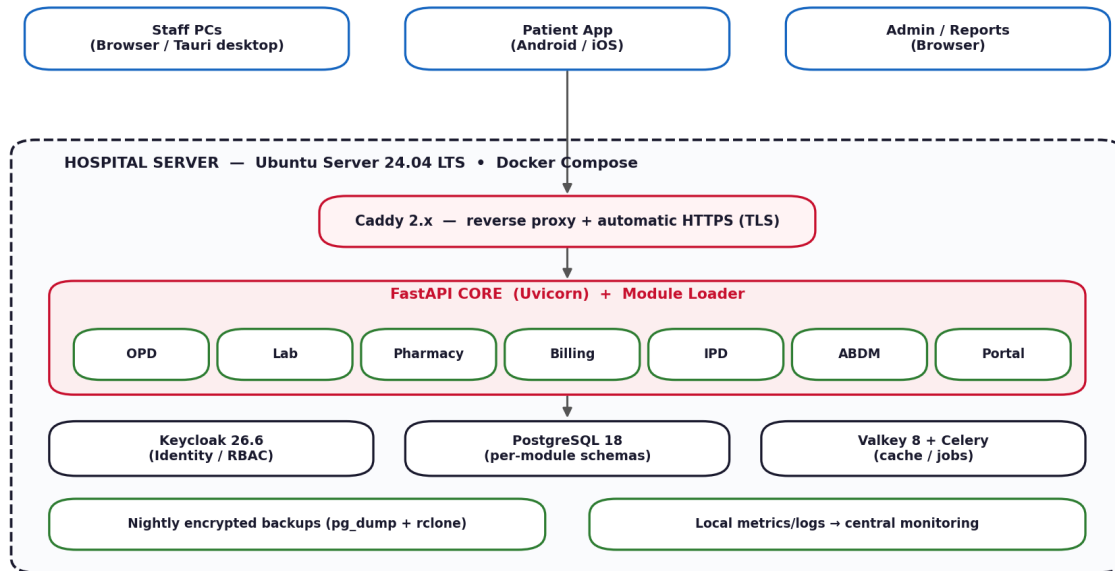


Figure 2 — Single-tenant deployment on one hospital server.

Minimum server specification (small clinic)

- 4-core CPU, 8 GB RAM, 256 GB SSD, Ubuntu Server 24.04 LTS; UPS-backed; nightly backup disk or NAS.
- Mid-size hospital: 8-core CPU, 16–32 GB RAM, 512 GB–1 TB SSD, RAID + off-site backup.

3. How the Package System Works

3.1 A package contains

- A backend Python wheel: a FastAPI router, its models, and Alembic migrations for its own tables.
- A frontend Vite Module-Federation 'remote': its screens and menu entries.
- A manifest (module.json): id, version, permissions, menu items, and depends_on.

3.2 Backend loading

The core discovers packages via Python `entry_points` and manages hooks with `pluggy`. On startup it reads enabled packages from a `module_registry` table, resolves dependency order, runs each package's pending Alembic migrations, mounts its router, and registers its permissions in Keycloak.

3.3 Frontend loading

The staff web app is a thin Vite shell that asks the core which packages are enabled, then dynamically loads each one's federated remote via `@originjs/vite-plugin-federation`, registering routes and menus at runtime. Not installed = no code, no menu.

3.4 Data isolation

One PostgreSQL database; each package owns a schema (e.g. `pharmacy.*`, `ipd.*`), migrations branched per package with Alembic. All packages share the core `patients` record for one unified patient view.

3.5 Distribution & the Module Store

Backend wheels and frontend bundles are published to a free self-hosted **Gitea** package registry. An admin-only Module Store screen installs/enables/disables packages, taking an automatic backup before any change.

4. Complete Tool Stack — Versions & Install Links

Version note: Versions below are the current stable releases as of July 2026. Pin the exact patch your package manager resolves at install time and re-check the official link, since patch releases (especially for pip/npm packages) are frequent. All tools are free and open-source; the only unavoidable costs are payment-gateway/SMS per-transaction fees, a domain name, and optional off-site backup storage.

4.1 System & infrastructure (install on the server)

Tool	Version	Purpose	Official link
Ubuntu Server	24.04 LTS	Server OS (26.04 LTS also available)	ubuntu.com/download/server
Python	3.13.x	Backend language	python.org/downloads
Node.js	24.x LTS	Frontend build & tooling	nodejs.org/en/download
PostgreSQL	18.4	Primary database	postgresql.org/download
Valkey	8.x	Cache / queue (BSD; free Redis fork)	valkey.io/download
OpenJDK (Temurin)	21 LTS	Runtime for Keycloak	adoptium.net
Keycloak	26.6.x	Identity, RBAC, SSO, MFA	keycloak.org/downloads
Docker Engine	latest stable	Containers	docs.docker.com/engine/install/ubuntu
Docker Compose	v2 (plugin)	Multi-container orchestration	docs.docker.com/compose/install
Caddy	2.x	Reverse proxy + auto HTTPS	caddyserver.com/docs/install
Git	2.4x	Version control	git-scm.com/downloads

4.2 Backend Python libraries (pip install)

Package	Version	Purpose	PyPI link
fastapi	latest 0.11x	API framework	pypi.org/project/fastapi
uvicorn[standard]	latest 0.3x	ASGI server	pypi.org/project/uvicorn
sqlalchemy	2.0.x	ORM	pypi.org/project/SQLAlchemy
alembic	1.14+	Migrations (branched per package)	pypi.org/project/alembic
pydantic	2.x	Validation	pypi.org/project/pydantic
pluggy	1.5+	Plugin/hook engine	pypi.org/project/pluggy
psycopg[binary]	3.2.x	PostgreSQL driver (psycopg3)	pypi.org/project/psycopg
celery	5.4+	Background tasks	pypi.org/project/celery
argon2-cffi	latest	Password hashing	pypi.org/project/argon2-cffi
python-jose[cryptography]	latest	JWT tokens	pypi.org/project/python-jose
python-multipart	latest	File uploads	pypi.org/project/python-multipart
fhir.resources	latest	FHIR R4 models	pypi.org/project/fhir.resources
weasyprint	latest	PDF (A4/thermal)	pypi.org/project/weasyprint
reportlab	latest	PDF generation	pypi.org/project/reportlab
openpyxl / pandas	latest	Excel & reports	pypi.org/project/openpyxl
pytest / pytest-asyncio / httpx	latest	Testing	pypi.org/project/pytest
locust	latest	Load testing	pypi.org/project/locust

4.3 Frontend & mobile (npm / tooling)

Package	Version	Purpose	Link
react / react-dom	19.x	UI library	react.dev
vite	7.x	Build tool / dev server	vite.dev
typescript	5.x	Typed JS	typescriptlang.org
@originjs/vite-plugin-federation	latest	Runtime module loading	npmjs.com/package/@originjs/vite-plugin-federation
antd	5.x	UI components	ant.design
@tanstack/react-query	v5	Data fetching/caching	tanstack.com/query
react-router-dom	v7	Routing	reactrouter.com

Package	Version	Purpose	Link
i18next / react-i18next	latest	Localisation	i18next.com
recharts / dayjs / axios	latest	Charts / dates / HTTP	recharts.org
Expo (React Native)	SDK 53+	Patient mobile app	expo.dev
Tauri	2.x	Desktop wrapper (needs Rust)	tauri.app
Rust (rustup)	latest	Tauri build dependency	rustup.rs

4.4 DevOps, testing, monitoring & compliance

Tool	Version	Purpose	Link
GitHub Actions	(hosted)	CI/CD (free tier)	github.com/features/actions
Gitea	1.2x	Self-hosted repo + package registry (Module Store)	about.gitea.com
Playwright	latest	End-to-end tests	playwright.dev
Prometheus	3.x	Metrics	prometheus.io/download
Grafana	12.x	Dashboards	grafana.com/grafana/download
Loki	3.x	Log aggregation	grafana.com/oss/loki
Uptime Kuma	1.23+	Uptime monitoring	github.com/louislam/uptime-kuma
GlitchTip	latest	Error tracking (self-host Sentry-compatible)	glitchtip.com
rclone	latest	Encrypted off-site backup sync	rclone.org/downloads
HAPI FHIR	latest	FHIR validation/server	hapifhir.io
ABDM Sandbox	(service)	ABDM integration & certification	sandbox.abdm.gov.in

5. Environment Setup — Install Order

A practical order for the two developers to set up a workstation and the eventual server.

1. Install Git, Python 3.13, and Node.js 24 LTS on each developer machine.
2. Install Docker Engine + Docker Compose (this runs PostgreSQL, Valkey, Keycloak, and the app locally).
3. Clone the repo; create a Python virtual environment and `pip install` the backend libraries (Section 4.2).
4. `npm install` the frontend libraries (Section 4.3) inside the Vite shell and each package.
5. Run `docker compose up` to start PostgreSQL 18, Valkey 8, and Keycloak 26.6 locally.
6. Apply Alembic migrations; start Uvicorn (core) and the Vite dev server.
7. For the server: provision Ubuntu 24.04, install Docker, deploy the Compose stack behind Caddy, point DNS, and let Caddy issue HTTPS.

Example (backend libs): `pip install fastapi "uvicorn[standard]" sqlalchemy alembic pydantic pluggy "psycpg[binary]" celery argon2-cffi "python-jose[cryptography]" python-multipart`

6. Phase-by-Phase Roadmap

Front-load the core + package framework (Phase 1), then ship each department as a package. A pilot-ready system (core + OPD + Appointments) lands around month 4; later packages are independent and can be reordered to match your design-partner.

Phase 0: Discovery & Validation (Wk 1–2, ongoing)

Goal — Confirm launch vertical; sign a design-partner.

- Interview 5–10 facilities; confirm which packages they'd enable first.
- Pick ONE launch vertical; sign a design-partner.
- Validate core + OPD/Appointments as mockups (Penpot).

Exit criterion: At least one facility commits to piloting.

Phase 1: Core Platform + Package Framework (Wk 1–6)

Goal — Build the core AND the plugin loader — the foundation.

- Core: identity/RBAC (Keycloak), patient master (UHID/ABHA), audit & consent, billing engine.
- Module Registry & Loader: manifest, dependency resolver, per-package migrations, feature flags.
- Vite shell loading federated remotes; CI (lint+test+build) on every commit; encryption + audit middleware.

Exit criterion: A trivial 'hello' package installs, enables, and disables end-to-end.

Phase 2: OPD + Appointments (MVP) (Wk 6–12)

Goal — Prove the package model and reach a sellable pilot.

- OPD package: ICD-10 diagnosis, notes, prescriptions, A4/thermal print.
- Appointments & Queue package.
- Both as installable packages with tests and menus.

Exit criterion: ★ MVP (~Month 4): design-partner runs a full OPD visit; packages install via the loader.

Phase 3: Lab & Imaging Package (Wk 12–15)

Goal — Add lab ordering/results on top of OPD.

- LOINC-coded orders, result upload, link to consultation and billing.

Exit criterion: Enabling Lab adds its menus/permissions without touching other packages.

Phase 4: Pharmacy & Inventory Package (Wk 15–19)

Goal — Batch/expiry stock and dispensing.

- Medicine master (HSN/GST), batch & expiry, auto-deduct on dispense, alerts; feed costs to billing.

Exit criterion: Dispensing deducts the right batch and raises alerts; costs appear on the bill.

Phase 5: Advanced Billing & Payments Package (Wk 19–23)

Goal — GST billing, payments, reports.

- Aggregate charges into GST bills; cash/UPI-QR/gateway; receipts (A4/thermal); revenue reports.

Exit criterion: A visit produces a correct GST bill and receipt; daily report reconciles.

Phase 6: ABDM / FHIR Package (Wk 23–29)

Goal — National-standard compliance — the key edge.

- HFR/HPR registration; ABHA create/verify; FHIR R4 bundles; Consent Manager; production certification.

Exit criterion: Sandbox milestones pass and bundles validate against the India FHIR profile.

Phase 7: Patient Portal & Mobile Package (Wk 29–34)

Goal — Patients see their own doctor-authored records (no AI-generated advice).

- Secure login (OTP/ABHA); own records; care instructions in plain + regional language; booking; bill access.

Exit criterion: A patient sees only their own records on web and app.

Phase 8: IPD / Inpatient Package (Wk 34–40)

Goal — A true hospital system with beds and admissions.

- Ward/bed board; ADT; nursing notes & vitals; room charges to billing; discharge summary as FHIR.

Exit criterion: Admit → move bed → charge → discharge with summary works end to end.

Phase 9: Analytics, Specialty & Localisation (Wk 40–44)

Goal — Purpose-built feel that beats generic HMS.

- Analytics dashboards; specialty packs for the launch vertical; regional-language pack; per-hospital branding.

Exit criterion: The launch-specialty workflow is measurably faster than a generic flow in a timed test.

Phase 10: Module Store, Deployment & Hardening (Wk 44–48)

Goal — Cheap to distribute, safe to run.

- In-app Module Store (Gitea registry); one-command install; pre-update backups + rollback; central monitoring.
- Security pass (OWASP, dependency scanning, pen self-test); package the Tauri desktop build.

Exit criterion: Bare Ubuntu → running, monitored, backed-up install (with the Store) in under an hour.

Phase 11: Pilot Rollout & Launch (Wk 48–52)

Goal — Prove it live; convert to a paying reference.

- Install core + chosen packages at the design-partner; training; install checklist.
- Load & security testing; fix real-world issues; testimonials; convert to paying reference; sales collateral.

Exit criterion: Pilot is live, paying, and a reference for the next sale.

7. Compliance, Operations & Business Model

7.1 Compliance (in the core, inherited by every package)

- DPDP: encryption at rest (LUKS + pgcrypto) and in transit (TLS via Caddy); API-layer RBAC; export/erase; consent capture.
- Audit: append-only audit log via core middleware across all packages.
- ABDM/FHIR: coded clinical fields and ABHA fields in the core; ABDM package handles ABHA/consent/FHIR against the free sandbox.

7.2 Operations (sustainable for two people)

- Identical Docker Compose install everywhere; only enabled packages differ.
- Central monitoring (Prometheus/Grafana + Uptime Kuma + GlitchTip) across all installs.
- Automated encrypted nightly backups with tested restore; backup before every install/update; one-command rollback.
- Per-package versioning: update one package for one hospital without touching the rest.

7.3 Business model — sell the core, charge per package

- Recurring core subscription (updates, support, backups) — never a one-time licence with free support.
- Per-package fees: hospitals pay only for departments they enable; the feature-flag list is both the licence and the technical switch.
- One-time implementation/training/migration fee per install; support tiers (business-hours vs priority).

8. Timeline Summary

Phase	Weeks	Month	Deliverable (package unless noted)
0 Discovery	1–2	1	Signed design-partner
1 Core + framework	1–6	1–2	Working plugin loader + core
2 OPD + Appointments	6–12	2–4	★ Pilot-ready MVP
3 Lab & Imaging	12–15	4	Lab package
4 Pharmacy	15–19	4–5	Pharmacy package
5 Billing & Payments	19–23	5–6	Billing package
6 ABDM / FHIR	23–29	6–7	ABDM package
7 Patient Portal + App	29–34	7–8	Portal + app
8 IPD	34–40	8–10	IPD package
9 Analytics/specialty/lang	40–44	10–11	Differentiation packages
10 Store/deploy/harden	44–48	11–12	Store + 1-hour install
11 Pilot & launch	48–52	12	Live paying reference

Technical and go-to-market plan, not legal or financial advice. Tool versions are current as of July 2026 — pin exact patches at install time and re-check official links. Verify current ABDM/NHA certification requirements before production.